

第 17 章

计划测试工作

本章是第五部分“使用测试文档”的开始。到目前为止，本书已给出了软件测试的概貌，并且寻找软件缺陷的基础也讲述过了——到哪里去找、如何测试、如何有效地测试。第五部分各章将把这些知识联系起来，说明和软件测试有关的所有工作是如何计划、如何组织以及如何和项目小组之间进行交流的。

提示 不要忘了软件测试员的目标是：

软件测试员的目标是尽可能早地找出软件缺陷，并保证其得以修复。

利用精心组织的测试计划、测试用例和测试报告，对测试工作进行正确的记录以及交流，将使达到目标变得更有可能是。

本章着重讲述测试计划，即工作中会遇到的最基本测试文档。测试新手一般不会被安排为项目建立全面测试计划——这是由测试负责人或者经理来做，而测试员一般是协助建立测试计划，因此需要了解计划测试工作包括哪些内容，测试计划需要哪些信息。通过这种方式，测试员为计划过程做出了努力，并利用掌握的信息组织自己的测试任务。此外，过不了多久就可以编制自己的软件测试计划了。

本章重点包括：

- 测试计划的目的
- 为什么计划是一个过程？而不是一个结果，有什么重要性
- 计划过程中需要考虑的问题
- 测试新手在测试计划中的作用

17.1 测试计划的目标

测试过程不可能在真空中进行。如果程序员编写代码而不说明它干什么、如何工作、何时完成，执行测试任务就很困难了。同样，如果测试员之间不交流计划测试的对象，需要什么资源，进度如何安排，整个项目就很难成功。软件测试计划（software test plan）是软件测试员与产品开发小组交流意图的主要方式。

IEEE 829-1998关于软件测试文档（Software Test Documentation）^①的标准以如下方式表达软件测试计划的目的：

① IEEE 829-1998标准可以从standards.ieee.org站点购买到电子或打印文档。

规定测试活动的范围、方法、资源和进度；明确正在测试的项目、要测试的特性、要执行的测试任务、每个任务的负责人，以及与计划相关的风险。

根据该定义和IEEE的其他标准，我们注意到测试计划采用的形式是书面文档。这虽然并不奇怪，但却是非常重要的点，因为尽管最终结果只是一页纸（或者联机文档，或者网页），但是这页纸不是测试计划的全部内容。

测试计划只是创建详细计划过程的一个副产品，重要的是计划过程，而不是产生的结果文档。

本章的标题是“计划测试工作”，而不是“撰写测试计划”。两者的区别显而易见。撰写的测试计划通常最终成为了一个空架子——束之高阁的文档（shelfware），以后再不会有人看。如果计划工作的目标从建立文档转移到建立过程，从撰写测试计划转移到计划测试任务，就不存在空架子问题了。

这并不是说描述和总结计划过程结果的最终测试计划文档就不需要了，相反，仍然需要有一个测试计划作为参考和文档归档——在一些行业这是法律的要求。重要的是计划是个副产品，并非计划过程的根本目的。

测试计划过程的最终目标是交流（而不是记录）软件测试小组的意图、期望，以及对将要执行的测试任务的理解。

如果和项目小组一起，花一些时间研究本章下面所列的主题，确保所有人都了解测试小组的计划，最终就可以达到这个目标。

17.2 测试计划主题

许多软件测试的书籍介绍了测试计划模板或者样本测试计划，可以随意修改，建立针对具体项目的测试计划。该方法的问题是容易把重点放在文档上，而不是计划过程上。大型软件项目的测试负责人和经理拿起测试模板的副本或者原有测试计划，花上几个小时剪切、复制、查找和替换，从而得到当前项目的“独特”测试计划。他们认为自己做了一件大事，仅用几个小时就建立了其他测试员要花几周或者几个月才能建立的测试计划。然而，他们没有抓住重点，当产品小组中没有人知道测试员在干什么，或者为什么那么干的时候，测试项目的弊端就显露出来了。

因此，本书中看不到测试计划模板。取而代之的是要遵循一系列重要主题的清单，该清单应该在整个项目小组——包括所有测试员中全面讨论、相互沟通并达成一致。该清单也许不能完全适用所有项目，但是因为它列出了常见的与重要测试相关的问题，所以比测试计划模板更实用。由于从本质上讲计划是一个动态过程，因此如果发现列出的问题不适应具体情况，就可以自行调整。

当然，测试计划过程的结果是某一种文档。如果行业或者公司有自己的标准，格式可以预先定义。软件测试文档的IEEE标准829-1998推荐了一种常用格式。除此之外，格式可由项目小组来决定，应该能够非常有效地交流工作成果。

17.2.1 高级期望

测试过程中的第1个论题是定义测试小组的高级期望。虽然这是项目小组全体成员必须一

致同意的基本论题，但是常常被忽视。它们可能被认为“太过明显”，并且想当然地假定每个人都了解——但是，优秀的测试员知道永远不要假定任何事。

- 测试计划过程和软件测试计划的目的是什么？测试员本人知道进行测试计划的理由——好吧，就算很快会知道——但是程序员知道吗？技术文档作者知道吗？管理部门知道吗？更重要的是，他们同意和支持这个测试计划过程吗？
- 测试的是什么产品？当然你相信它是Ginsumatic v8.0版，但是真能够确定吗？这个v8.0版计划是完全重写的还是仅仅维护升级的？它是一个独立程序还是由几千个小程序组成的？它是自行开发的还是第三方开发的？还有，Ginsumatic究竟是什么东西？

为了使测试工作成功，必须完全了解产品是什么，以及其数量和适用范围。从说明书得到产品描述是一个好的开端，但是将其给小组其他人看就会发生奇怪的现象。测试员并不希望程序员宣称：“我写的代码不执行这样的功能！”

- 产品的质量和可靠性目标是什么？这是大家争论的焦点，但是使所有人一致同意这些目标是绝对必要的。销售代表会说软件运行要尽量快；程序员会说软件要使用最酷的技术；产品支持会说软件不能有引起任何冲突的缺陷。他们不可能都对。怎样权衡快和酷？怎样向产品支持工程师解释软件有可能携带导致崩溃的缺陷？由于测试小组将会测试产品的质量和可靠性，因此要知道目标是什么；否则怎么知道软件是否达到目标呢？测试计划过程的结果必须是清晰、简洁的、在产品质量和可靠性目标上一致通过的定义。目标必须绝对，以免说不清是否达到目标。如果销售人员想快，就让他们定义基准——能够每秒钟处理一百万个事务，或者比对手XYZ执行同样的任务快一倍。如果程序员喜欢出色的技术，就说清技术是什么。不要忘了，不成熟的技术就是软件缺陷。至于软件缺陷，不能保证全能找到——这是不可能的。但是，可以说目标是自动猴子测试员运行24小时不崩溃，或者执行全部测试用例而找不到新的软件缺陷，等等。要有针对性。随着产品发布日期临近，质量和可靠性目标不应该再有不同的意见了。每一个人都应该知道这一点。

17.2.2 人、地点和事

测试计划需要明确在项目中工作的人，他干什么，怎样和他联系。在小项目中这似乎没有必要，但是即使是小项目，小组成员也可能分散在很远的地方，或者人员变动，为跟踪谁做什么造成困难。大型团队可能有数十或数百个联络点，测试小组很可能要和所有人打交道，知道他们是谁和如何联系是非常重要的。测试计划应该包括项目中所有主要人员的姓名、职务、地址、电话号码、电子邮件地址和职责范围。

同样，文档存放在哪里（测试计划放在哪个文件夹或服务器上），软件从哪里可以下载，测试工具在哪里等都需要明确。考虑邮件地址、服务器和网站地址。

如果在执行测试时硬件不可缺少，那么它放在哪里、如何得到？如果有进行配置测试的外部测试实验室，那么它们具体在哪里？它们的进度是怎样安排的？

该论题最好的描述是“测试新手问题指南”。这通常是要测试新手负责的好的测试计划部分。找到所有问题的答案，并把发现记录下来。你想知道的也是别人想知道的。

17.2.3 定义

让项目小组中的全部成员在高级质量和可靠性目标上达成一致是一件困难的事情。不幸

的是,这只是软件项目中需要定义的用词和术语的开始。回顾第1章“软件测试的背景”中关于软件缺陷的定义:

- 1) 软件未实现产品说明书要求的功能。
- 2) 软件出现了产品说明书指明不应该出现的错误。
- 3) 软件实现了产品说明书未提到的功能。
- 4) 软件未实现产品说明书虽未明确提及但应该实现的目标。

能确认小组全部成员知道、理解——更重要的是——同意该定义吗?项目经理知道软件测试员的目标吗?如果不是这样,测试计划的过程就是保证他们要理解和同意。

这是项目小组中最大的问题之一——忽视在开发产品中这些常用术语应用时的含义。程序员、测试员和管理部门对术语都有各自的理解。如果程序员和测试员对软件缺陷是什么的基本理解未能达到一致,可以想见争执在所难免。

测试计划过程就是定义小组成员的用词和术语。对差异要进行鉴别,并得到一致的同意,使全体人员说法一致。

以下是一些常用术语和相当松散的定义。不要把它当做完整或者定义明确的清单。它取决于具体项目、开发小组遵循的开发模式,以及小组成员的经验。该清单列出的术语只是在应该为项目定义哪些内容的考虑上开拓思路,并说明使全体人员了解其含义的重要性。

- **构造。**程序员放在一起需要测试的代码和内容的搜集。测试计划应该定义构造的频率(每天、每周等)以及期望的质量等级。
- **测试发布文档 (TRD)。**程序员发布的文档。对每一个构造都声明新特性、不同特性、修复问题和准备测试的内容。
- **Alpha版。**意在对少数主要客户和市场进行数量有限的分发,用于演示目的的早期构造。其无意在实际环境中使用。使用alpha版的所有人员必须了解确切内容和质量等级。
- **Beta版。**意在向潜在客户广泛分发的正式构造。回顾第16章“缺陷轰炸和beta测试”所述,进行beta测试的原因需要定义。
- **说明书完成。**说明书预计完成并且不再更改的日程安排。干过几个项目之后,就会知道这个期限只能在玄幻小说中实现,但是它确实应该设定,以后只能进行控制范围内的小改动。
- **特性完成。**程序员不再向代码增加新特性,并集中修复缺陷的日期安排。
- **软件缺陷会议。**由测试经理、项目经理、开发经理和产品支持经理组成的团队,每周召开会议审查软件缺陷,并确定哪些需要修复,应该如何修复。软件缺陷会议是在测试计划中建立质量和可靠性目标的主要措施之一。

17.2.4 团队之间的责任

团队之间的责任是明确指出可能影响测试工作的任务和交付内容。测试小组的工作由许多其他功能团队驱动——程序员、项目经理、技术文档作者等。如果责任未明确,整个项目——尤其是测试——就会出现戏剧化情景:“我拿了,不,你拿了,你还没有处理,不,我想你已经处理过了”,从而导致重要的任务被忘记。

需要定义的任务类型不像测试员的测试、程序员的程序那样容易分清。麻烦的任务可能有多个负责者,有时没有责任者,或者由多人共同负责。计划这些任务和交流计划最容易的

方法是使用如图17-1所示的简表。

任务	程序管理	程序员	测试	技术文档作者	营销人员	产品支持
撰写产品版本声明	---				X	
创建产品组成部分清单	X					
创建合同	X				---	
产品设计/功能划分	X			---	---	
项目总体进度	X	---		---	---	
制作和维护产品说明书	X					
审查产品说明书	---	---	---	---	---	---
内部产品的体系结构	---	X				
设计和编写代码		X				
测试计划			X			
审查测试计划		---	X	---	---	---
单元测试		X				
总体测试			X			
创建配置清单		---	X		---	---
配置测试			X			
定义性能基准	X		---			
运行基准测试			X			
内容测试			---	X		
来自其他团队的测试代码			---			
自动化/维护构建过程		X				
磁盘构建/复制		X				
磁盘质量保证			X			
创建beta测试清单					X	---
管理beta程序	---		---		X	---
审查印刷的资料	---	---	---	X	---	---
定义演示版本	---				X	
生成演示版本	---				X	
测试演示版本			X			
缺陷会议	X	---	---		---	---

图17-1 借助表格的帮助来组织团队之间的责任

表中任务列于左边，可能的责任者横列在顶部。“×”号表示任务的责任者，短线(-)表示任务的参加者，空白表示团队不负责该任务。

确定表格列出哪些任务取决于经验。理想情况下，小组中有资深成员可以先过一遍清单，但是每一个项目是不同的，有其独特的团队之间的责任和依赖关系。良好的开端是找人询问过去项目的情况，以及能记得的曾被疏忽的任务。

17.2.5 哪些要测试，哪些不要测试

有时会惊奇地发现软件产品中包含的某些内容不必测试。这些内容可能是以前发布过或者测试过的软件部分。来自其他软件公司并已经测试过的内容可以直接接受。外包公司会提供预先测试过的产品部分。

计划过程需要验明软件的每一部分，确定它是否要测试。如果没有测试，需要说明这样做的理由。如果由于误解而使部分代码在整个开发周期漏掉，未做任何测试，就可能导致一场灾难。

17.2.6 测试的阶段

要计划测试的阶段，测试小组就会查看预定的开发模式，并决定在项目期间是采用一个测试阶段还是分阶段测试。在边写边改模式中，可能只有一个测试阶段——不断测试，直到某个成员宣布测试停止。在瀑布和螺旋模式中，从检查产品说明书到验收测试可能有几个阶段，测试计划也属于其中一个测试阶段。

测试的计划过程应该明确每一个预定的测试阶段，并告知项目小组。该过程一般有助于整个小组形成和了解全部开发模式。

注意 与测试阶段相关联的两个重要概念是进入和退出规则。测试小组不能只是周一来上班，看看日历就知道该进入下一个阶段了。每一个阶段都必须有客观定义的规则，明确地声明本阶段结束，下一阶段开始。

例如，说明书审查阶段可能在正式说明书审查公布时结束。beta测试阶段可能在测试员完成验收测试，从预定beta测试构造中没有发现新的软件缺陷时开始。

假如没有明显的进入和退出规则，测试工作就会变成单一的，且毫无头绪的工作——很像边写边改开发模式。

17.2.7 测试策略

与定义测试阶段相关联的练习是定义测试策略。测试策略描述测试小组用于测试整体和每个阶段的方法。回顾到目前为止所学的软件测试知识。如果你面对需要测试的产品，就需要决定使用黑盒测试好，还是白盒测试好。如果决定综合使用这两种技术，那么在软件的哪些部分，什么时候运用它们呢？

某些代码用手工测试，而其他代码用工具和自动化测试也许是个不错的想法。如果要使用工具，那么是否需要开发，或者能够买到已有的商用解决方案？如果是，选择哪一种情况？也许更有效的方法是把整个测试工作外包到专业测试公司，只要形同虚设的测试员监督他们的工作即可。

做决策是一项复杂的工作——需要由经验相当丰富的测试员来做，因为这将决定测试工作的成败。使项目小组全体成员都了解并同意预定计划是极其重要的。

17.2.8 资源需求

计划资源需求是确定实现测试策略必备条件的过程。在项目期间测试可能用到的任何资源都要考虑到。例如：

- **人员。**人员数量、经验和专长？他们是全职、兼职、合同还是学生？
- **设备。**计算机、测试硬件、打印机、工具。
- **办公室和实验室空间。**在哪里？有多大？如何布局？
- **软件。**文字处理程序、数据库程序和自定义工具。要购买哪些东西？要写什么材料？
- **外包测试公司。**用他们吗？选择他们有什么原则？他们的费用如何？
- **其他配备。**磁盘、电话、参考书、培训资料。在项目期间还需要别的吗？

特定资源需求取决于项目、小组和公司，因此测试计划工作要仔细估算测试软件的要求。开始不做好预算，到项目后期获取资源通常很困难，甚至无法做到，因此创建完整清单是必要的。

17.2.9 测试员的任务分配

一旦定义了测试阶段、测试策略和资源要求，这些信息加上产品说明书就可以分配每个测试员的任务。前面讨论的团队之间的责任是指哪些功能性团体（管理、测试和程序员等）负责哪些高级任务。计划测试员任务分配是指，明确测试员负责软件的哪些部分、哪些可测试特性。表17-1给出了一个极为简化的Windows写字板程序的测试员任务分配表。

表17-1 写字板程序的高级任务分配

测 试 员	测试任务分配
Al	字符格式：字体、大小、颜色、样式
Sarah	布局：项目符号、段落、制表位、换行
Luis	配置和兼容性
Jolie	用户界面：易用性、外观、辅助特性
Valerie	文档：在线帮助、滚动帮助
Ron	压力和负载

实际责任表更加详细，确保软件的每一部分都分配有人测试。每一个测试员都会清楚地知道自己负责什么，而且有足够的信息开始设计测试用例。

17.2.10 测试进度

测试进度需要以上所述的全部信息，并将其映射到整个项目进度中。该阶段一般在测试计划工作中至关重要，因为原以为很容易设计和编码的一些必要特性可能后来证实测试非常耗时。一个例子是某程序在不明显的有限区域之外不执行打印。没有人意识到打印对测试的影响，而在产品中保留该特性，结果导致打印机配置测试要花几周时间。作为测试计划的一部分，完成测试进度安排可以为产品小组和项目经理提供信息，以便更好地安排整个项目的进度。他们甚至会根据测试进度决定砍掉产品的一些特性，或者将其推迟到下一个版本中推出。

关于测试计划的一个重要问题是测试工作通常不能平均分布在产品开发周期中。有些测试以说明书和代码审查、工具开发等形式在早期进行，但是测试任务的数量、人员的数量和测试花费的时间随着项目的进展不断增长，在产品发布之前会形成短期的高峰。图17-2显示了典型的测试资源图表。

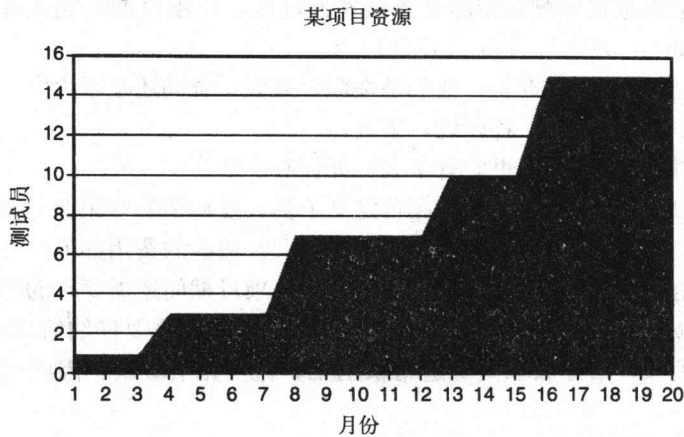


图17-2 项目中的测试资源数目随着开发的进度而增长

持续增长的结果是测试进度受到项目中先前事件的影响越来越大。如果项目中某一部分交给测试组时晚了两周，而按照进度只有3周测试时间，结果会怎样？把3周的测试在1周内进行，还是把项目推迟2周？这个问题称为进度破坏（schedule crunch）。

使测试任务摆脱进度破坏的一个方法是测试进度避免定死启动和停止任务的日期。表17-2是肯定会使小组陷入进度破坏的测试进度。

表17-2 固定日期的测试进度

测试任务	日期
测试计划完成	3/5/2001
测试用例完成	6/1/2001
通过第1轮测试	6/15/2001-8/1/2001
通过第2轮测试	8/15/2001-10/1/2001
通过第3轮测试	10/15/2001-11/15/2001

相反，如果测试进度根据测试阶段定义的进入和退出规则采用相对日期，那么显然测试任务依赖于其他先完成的可交付内容。单个任务需要多少时间也很明显。表17-3给出了一个例子。

表17-3 采用相对日期的测试进度

测试任务	开始日期	使用时间
测试计划完成	说明书完成后7天	4个星期
测试用例完成	测试计划完成	12个星期
#1测试通过	代码完成构建	6个星期
#2测试通过	Beta版构建	6个星期
#3测试通过	发行版构建	4个星期

许多软件进度安排产品会使该过程容易管理。项目经理或者测试经理最终负责进度安排，可能会使用此类软件，但是要求测试员参与安排自己的具体任务。

17.2.11 测试用例

本书前面已经讲过什么是测试用例了。第18章“编写和跟踪测试用例”将进一步讲述细节。测试计划过程将决定用什么方法编写测试用例，在哪里保存测试用例，如何使用和维护测试用例。

17.2.12 软件缺陷报告

第19章“报告发现的问题”将讲述用于记录和跟踪所发现的软件缺陷的技术。报告的各种可能的方式包括：隔着墙壁呼喊，使用粘性便签，使用复杂的缺陷跟踪数据库。使用哪些过程需要计划，以便每个软件缺陷从发现到修复的过程中都被跟踪——这样就绝不会被忘掉。

17.2.13 度量和统计

度量和统计是跟踪项目发展、成效和测试的手段。详情见第20章“成效评价”。测试的计划过程应该明确收集哪些信息，要做什么决定，谁来负责收集。

实用的测试度量的例子如下：

- 在项目期间每天发现的软件缺陷总数。
- 仍然需要修复的软件缺陷清单。
- 根据严重程度对当前软件缺陷评级。
- 每个测试员找出的软件缺陷总数。
- 从每个特性或者区域发现的软件缺陷数目。

17.2.14 风险和问题

测试计划中常用而且非常实用的部分是明确指出项目的潜在问题或者风险区域——这是对测试工作有影响的地方。

假设有十几个测试新手，全部软件测试经验来自于阅读本书，受命测试新建核电站的软件，这就是风险。也许某个新软件要对1500个调制解调器进行测试，在项目进度中的时间只能对其中500个进行测试，这又是一个风险。

软件测试员要负责明确指出计划过程中的风险，并与测试经理和项目经理交换意见。这些风险应该在测试计划中明确指出，在进度中给予说明。有些是真正的风险，而有些最终证实是无关紧要的。重要的是尽早明确指出，以免在项目晚期发现时感到惊慌。

17.3 小结

即使对于小型项目，开发测试计划也是不可轻视的大任务。不错，填写模板的空白项很容易，几个小时就可以打印出测试计划的副本，但是这样没有抓住要点。测试计划是一项全体测试员和整个产品小组中的主要人员参与的工作。做好测试计划要花费几周甚至几个月的时间。但是在产品开发早期，对为什么要测试，怎样去测试形成全面的理解和一致的同意，就会使测试工作更加顺利地展开，而不是赶进度。注意，重要的是创建测试计划的过程而不

是结果文档。

如果你是测试新手——也许在读本书的时候是——你也许不会负责开发整个软件测试计划。然而，你应该准备着向测试负责人或者经理为本章所列的所有论题提供输入内容。你要负责测试软件的某些方面和特性；你制订的进度、所需的资源和承担的风险最终全部要融入整个测试计划中。

17.4 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. 测试计划的目的是什么？
2. 为什么创建计划的过程是关键，而不是计划本身？
3. 为什么定义软件的质量和可靠性目标是测试计划的重要部分？
4. 什么是进入和退出规则？
5. 列出在测试计划时应该考虑的一些常用测试资源。
6. **判断是非：**制订进度要符合固定日期，以使测试任务或者测试阶段何时开始，何时结束没有异议。